QUANTIZATION METHODS FOR RETRIEVAL AUGMENTED SYSTEMS

Bao Nguyen [*1] **Kaushal Gumpula** [*1] **James Zhao** [*1]

ABSTRACT

Retrieval Augmented Generation (RAG) systems are critical to a variety of natural language processing applications. This project explores the bottlenecks that naturally occur in the retrieval stage of large-scale RAG systems. We identify three main bottlenecks in the retrieval pipeline: the embedding model latency, memory overhead of embedded documents, and latency of the semantic search to retrieve the relevant documents. Our experiments, conducted on the MS-MARCO dataset, show comprehensive results on different techniques to mitigate these bottlenecks. We provide insights on model quantization and the effects of jointly combining different embedding reductions, specifically on their overlooked impact on semantic search efficiency.

1 INTRODUCTION

Training large language models (LLMs) on massive amounts of data has allowed the development and rapid deployment of generative models with impressive reasoning abilities. However, these black-box systems are prone to hallucinations, and do not account for the latest data in their parametric memory. To address these issues, Retrieval Augmented Generation (RAG) (Lewis et al., 2021) has grown in popularity, allowing the model to rely on external knowledge to give more accurate answers. This external knowledge is stored in a database as embeddings (i.e. vectorization of a raw modality) on which the similarity with a given query can be evaluated. Many companies (OpenAI, Cohere) and research communities (Reimers & Gurevych, 2019) have released their own embeddings, amid a competition to make them semantically grounded and to make the retrieval of sources scalable and efficient.

Many state-of-the-art models produce embeddings with 1024 dimensions, each of which is encoded in float32 (4 bytes per dimension). Intuitively, as the embedding dimension grows, the cost of storage increases alongside the amount of information stored in the latent representations. Thus, there is a bottleneck when it comes to naively scaling the retrieval to support a large-scale knowledge-base: in terms of memory (embedding storage) and latency (time to retrieve).

Thus, our main contributions involve a thorough analysis of end-to-end retrieval systems, and tackle optimizations in the bottlenecks in a modular fashion. Specifically, our contributions shed light on the following research questions:

Model Quantization: How well do quantization methods scale to smaller models on large knowledge bases?

Dimensionality Reduction: What are the realistic limits of dimensionality reduction, and are there ways to improve these limits?

Embedding Quantization: What is the optimal embedding quantization / dimensionality reduction tradeoff to still maximize performance and minimize search latency? How do these methods affect the efficiency of the final search strategy?

2 PROBLEM

RAG pipelines are usually broken down into two parts: retrieval and generation. The generation is often done by an LLM given the query concatenated with external knowledge, but there exists a memory bottleneck in storing the embeddings, as well as an inherent search latency. In practice, these problem are exacerbated when LLM systems have high-frequency queries (e.x. short-form generation) or a large knowledge base.

When it comes to the retriever model, we want to improve the latency of encoding new queries/documents, as every query will have to be encoded during the retrieval pipeline. By using quantization methods on the retriever model, we expect to reduce the latency of generating embeddings during RAG.

When it comes to the size of the vector database, two primary methods exist for reducing the memory overhead. The first is embedding quantization, which reduces the precision and thus memory usage of each dimension of each vector, and the second is dimensionality reduction, which directly reduces the number of dimensions stored by each vector. For a fixed memory budget, we also want to individually balance the tradeoffs between these two different methods in terms of retrieval accuracy and latency.

Thus, we provide a thorough analysis of optimizations through the end-to-end retrieval pipeline. To the best of our knowledge, each component we mentioned has been marginally studied, but there is a need for a more holistic study of how different sequential optimizations add up. If quantizing the embedding model is done once to obtain a primary heavy version of the corpus, the optimization of the latter through dimensionality reduction and quantization of the embeddings is rarely jointly studied, not to mention the consequences that these choices have on the efficiency of a given search method, and ultimately the retrieval accuracy. We aim at tackling this problem with a thorough cross-component analysis at the intersection of systems engineering and ML algorithmic optimizations.

3 Literature Review

3.1 Model Quantization

Quantization has become a widely adopted technique in Machine Learning that has risen out of necessity as foundation models grow larger in size. For resource-constrained platforms, it is difficult to natively support these models as even the "smaller" LLMs can reach billion-parameter scales. Quantization tackles this problem by representing model components in a lower-precision format, reducing memory overhead and accelerating inference. Generally, quantization methods fall into two main categories: *quantization-aware training* and *post-training quantization*.

Quantization-aware training (QAT) usually involves fine-tuning or completely retraining a foundation model. Recent research has even shown competitive performance with "1-bit" LLMs that use a ternary bit which can take values of $-1, 0, 1$. BitNet 1.58 was able to achieve this with vast improvements to latency, generative token throughput, and a large reduction in memory overhead with minimal performance loss (Ma et al., 2024). BitNet 1.58 achieves this by replacing fully connected layers in transformers with custom linear layers that rescale the weight matrices and then clip the values to the nearest ternary bit integer (Ma et al., 2024).

To further reduce the computational cost of quantization, a family of methods called **post-training quantization (PTQ)** has emerged, offering the benefits of quantization methods without needing to retrain the original foundation model. Additionally, PTQ methods offer the ability to do quantization when it is not possible to access the original model training pipeline or training data.

LLM.int8() uses a two-part quantization procedure by first introducing new scaling factors by using vector-wise quantization instead of just having a single scaling constant per tensor. This mitigates the impact of a single outlier strongly affecting the quantization precision of the other values (Dettmers et al., 2022). They also use a mixed-precision scheme for large-magnitude outliers with 16-bit precision instead of the 8-bit precision used for the standard values (Dettmers et al., 2022).

GPTQ does quantization layer-by-layer by constructing a least-squares objective to minimize the loss between the layer output with the original weights and the layer output with the quantized weights (Frantar et al., 2023). **GPTQ** is an example of a PTQ method that only quantizes the weights of the model.

Quantizing activations in LLMs has been found to be much more difficult due to very strong outlier channels in the activations, while the distribution of weights in these models is much more smooth. **SmoothQuant** works by introducing per-channel scaling factors to smooth out the input activations to mitigate the outlier channels (Xiao et al., 2024). Their approach actually makes quantizing the weights harder as it shifts the outlier channels to the weight distribution. However, this tradeoff is justified as the transformed weight distribution is still relatively easy to quantize (Xiao et al., 2024).

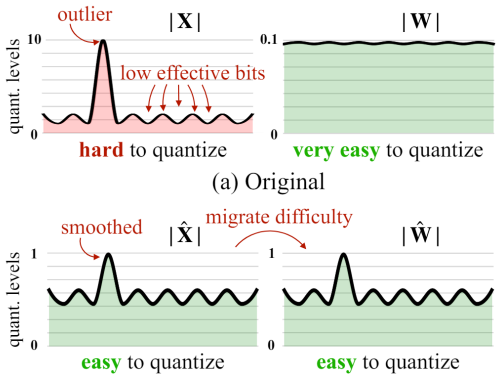


Figure 1. Migration of outlier channels in SmoothQuant.

The SmoothQuant paper describes a tradeoff between the difficulty of quantizing the activations and the difficulty of quantizing the weights. As most of the status quo results are on LLMs, we think it will be interesting to investigate the extension of these novel methods to retriever models.

3.2 Embedding Quantization

Similarity metrics like cosine similarity involve vector dot products which scales linearly with respect to the embedding vector size. Thus, this can become a bottleneck with large embedding vectors as retrieval speed starts to slow down and the memory overhead increases. To mitigate this, we can use quantization to turn our real-valued continuous embeddings into discrete values through *scalar quantization* (e.g. int8) or into 1-bit values with *binary quantization*. Specifically

for binary quantization, instead of cosine similarity, we have the added benefit of using the *Hamming distance* metric to efficiently retrieve the embeddings (Zhang et al., 2024).

In practice, scalar quantization involves binning and mapping the float values into a discrete range, requiring a calibration corpus of embeddings, where the min/max values of each dimension are used to compute the binning. Binary quantization is a binary mapping of each dimension floating values with thresholding (around 0 in practice).

However, a concern with this discretization is that it is not necessarily preserving the semantic information contained in the original embedding vector. An alternative approach involves using a simple AutoEncoder architecture to learn new embeddings that will preserve semantic information when thresholded (Zhang et al., 2024), but this is less systematic and harder to optimize.

3.3 Dimensionality Reduction

In addition to reducing the precision of the embeddings, we can also reduce the size or dimensionality of the embedding vectors themselves. As our embedding vectors can be represented as high-dimensional data points, we can use standard techniques to reduce the embedding space of dimensionality d that spans these points into a lower dimensional space of dimensionality k. In doing so, metrics that use dot-products can be sped up to be $O(k)$ instead of $O(d)$ where $k << d$.

Several techniques exist for unsupervised dimensionality reduction (for vector databases). Common linear dimensionality reduction methods include Principal Component Analysis (PCA), and nonlinear dimensionality reduction techniques include Uniform Manifold Approximation and Projection (UMAP) and t-Distributed Stochastic Neighbor Emulation (t-SNE). In this project, we mainly focus on using PCA to compress the embedding space.

3.4 Semantic Search for Retrieval

Given a query, the retrieval of relevant sources usually involves a K-nearest neighbor approach (KNN). The most computationally expensive methods for KNN involves a dense search, in which an exhaustive pairwise comparison of query and document embeddings is performed. To speed-up this procedure, it is standard to use an efficient approximate-nearest-neighbor implementation, such as FAISS (Douze et al., 2024) or ANNOY (Spotify, 2015), to perform semantic search. These indexing modules allow to easily experiment different search algorithms, exact or approximate (eg. *FAISS IndexIVF, IndexHNSW*). We do not detail them here, only when it is relevant in the method and results, and we refer to FAISS official documentation for the rest.

Example Query	Relevant Passage (Ground Truth)
"average number of lightning strikes per day"	"Although many lightning flashes are simply cloud -to-cloud, there are as many as 9,000,000 reported lightning strikes ..."

Table 1. Example Passage from MS-MARCO

4 Overview

As depicted in Figure 2, we have implemented a standard End-to-End retrieval system with several modular components. Our code is published here [1]:

Firstly, we have some large corpus, and a smaller number of user queries. Next, these documents are processed through an embedding model, with optional model quantization methods being applied to the model's outputs. Each query and passage results in one fixed-embedding vector.

Next, the corpus and query embeddings are stacked into a vector store (implemented via numpy arrays, though this can be abstracted to more tailored vector database systems), and various post-processing methods can be applied to the embeddings. In our experiments, we primarily applied truncation, Principal Component Analysis, and Embedding Quantization Methods.

Afterwards, retrieval is done by searching over the corpus embeddings, implemented using Facebook AI Similarity Search (FAISS), to find the nearest-document for every query vector. Both exact search and approximated search (Inverted File Index) allow for a tradeoff of speed and accuracy.

Lastly, using the top-1000 KNN results, we can obtain a ranking of the most related documents, and score the ranking according to the ground truth relevant document. We detail the dataset and scoring metric hereafter.

5 Methods

5.1 Experimental Setting

5.1.1 Benchmark

Across all experiments, we used the MS-MARCO (Bajaj et al., 2018) Passage-Ranking dataset, using the Full-Ranking Subtask. For this task, the corpus consists of 8,841,823 few-sentence passages, with 55,578 passages containing between 1-6 ground-truth passages. For the benchmark, our system is allowed to submit up to 1,000 docu-

[1] https://github.com/kgumpula2/sentence-transformers/tree/master

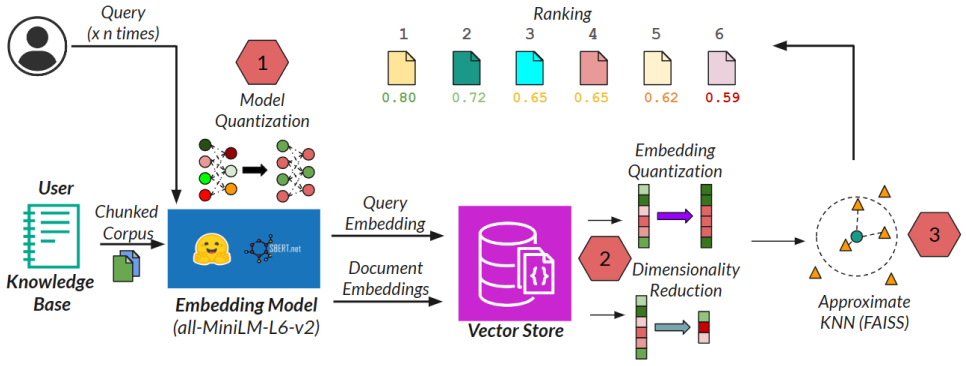


Figure 2. Overview of our Proposed System.

ments from among the 8.8 million passages, in which the final evaluation is performed over the MRR@1000. Datasets and evaluation code were obtained from the official benchmark documentation [2], and custom processing scripts were written by us. Experiments were run both using the full dataset, as well as subsets of queries and passages due to compute limitations.

5.1.2 *Models*

For our experiments, we use models from the Sentence-Transformer package [3]. Because we intend our results to be relatively model-agnostic, we used the `all-MiniLM-L6-v2` pretrained model for most of our experiments, which is a standard SBERT Model ranking 6th among Sentence-Transformer's pretrained models [4].

Each passage and query is passed through the embedding model to generate embeddings, and the corresponding sets of vectors are saved as corpus-embedding and query-embedding matrices, each being *(n,d)* matrices.

All models and downstream optimizations were run using Python Implementations / Wrappers of libraries. Additional Python+Bash driver & processing scripts were constructed by us.

5.2 Indexing and Searching

Searching and Indexing was implemented using the Python wrapper of Facebook AI Similarity Search (FAISS) [5]. Due to the large corpus size, approximate search methods (inverted file index, *IVF*) were used to allow for reasonable runtime. Unless otherwise specified, the search parameters used were `nlist`=10,000, `nprobe`=200, `knn`=1000, where `nlist` is the number of clusters the data is partitioned into, `nprobe` is the number of nearby clusters that are searched, and `knn` is the number of relevant documents that are returned by each search query. In 5.5, we experiment with different indexes to study the impact of memory alleviation for the different search methods.

Before running FAISS, model embeddings are L2-Normalized, such that computing the inner product is equivalent to computing vector cosine-similarity. All experiments are run using inner-product similarity, unless otherwise specified, and wall-clock timings were recorded for training (e.x. voronoi cluster fitting) and search stages.

5.3 Model Quantization

For our model quantization experiments, we used the `all-MiniLM-L6-v2` pretrained model as the baseline model. Linear Quantization was done on both GPU and CPU, where we used *quanto*[6] for GPU weight-only quantization and *PyTorch*[7] for CPU weight-only quantization. On the GPU, we tested int8, int4, int2 quantization precisions. On the CPU, we tested int8 quantization (the lowest precision PyTorch offers) for both static quantization, where weights are frozen before runtime, and dynamic quantization, where scaling factors are computed dynamically during inference. Due to resource constraints, the quantization experiments were run on a smaller subset of the data (107,400 documents on the GPU, 8,434 documents on the CPU). Additionally, because we do not use the full corpus, we use Sentence-Transformer's MS-Marco evaluation pipeline for testing on a small corpus, adapting the code to time throughput results and support quantization. Note that this is distinctly different from the other experiments where

[2] https://microsoft.github.io/msmarco/Datasets
[3] https://github.com/UKPLab/sentence-transformers
[4] https://www.sbert.net/docs/pretrained_models.html
[5] https://github.com/facebookresearch/faiss
[6] https://github.com/huggingface/quanto
[7] https://github.com/pytorch/pytorch

we use our own **custom** pipeline and implementation for evaluation and timing.

Because the official implementations of SmoothQuant only apply directly to specific LLM models, we used the *torchao*[8] library implementation. Specifically, we swapped the linear module in our baseline model with *torchao*'s implementation of SmoothQuant's linear module. As SmoothQuant is a dynamic quantization approach that is applied to both weights and activations, it is necessary to calibrate the quantized model for inference. Thus, we tuned the activation scaling factors on subsets of the underlying corpus from MS-Marco. After the tuning process, we froze the weights of the tuned model and tested using the saved quantized model.

5.4 Dimensionality Reduction

For our Dimensionality Reduction Experiments (in which the dimension of each vector was reduced from some value d to $k, k << d$), several methods were investigated.

5.4.1 Principal Component Analysis

Principal Component Anaylsis is a method of dimensionality reduction, in which the data is projected onto a set of k vectors that contain the most variance in a dataset. Implementation-wise, Principal Component Analysis can be accomplished by finding the eigenvalues and corresponding eigenvectors of the covariance matrix, and the first k vectors, reverse-sorted by eigenvalues, are used as as the principal components.

We investigated two difference methods of PCA. The first method is Scikit-Learn's Implementation (`sklearn.decomposition.PCA`) [9], in which a model is firstly trained using the corpus embeddings, and both the corpus and query embedding-matrices are projected using the learned PCA matrix and saved as new vector databases. Lastly, the search pipeline can be run simply as if the original embeddings had a reduced dimension d. Notably, performing this PCA operation tended to be extremely compute heavy, as each PCA step took between 1200-4500 seconds to compute principal components and project the data.

The second investigation method we investigated involved FAISS's built-in PCA operation. Mathematically, this library's implmentation is equivalent to scikit-learn's, but is computed noticeably faster than scikit-learn's implementation.

[8] https://github.com/pytorch/ao

[9] https://scikit-learn.org/stable/modules/generated/sklearn.decomposition.PCA.html

5.4.2 Matryoshka Embedding - Truncation

Matryoshka Embedding Models (Kusupati et al., 2024) utilize a specialized truncation-aware training paradigm to allow the embeddings to still perform well at higher truncation levels. In particular, for some embedding training objective $\ell(y, \hat{y})$ computed over a set of d-dimensional embedding x_d and labels y_d, the Matryoshka Learning Objective is defined as a weighted average of losses computed on truncated representations, $\sum_i \alpha_i \ell(\text{trunc}(y_d, d_i), f(\text{trunc}(x_d, d_i)))$.

Since Matryoshka Embedding Models only modify the training objective, the usage of these models for inference simply requires swapping out the embedding model. For these experiments, we use two pretrained Sentence Transformer models, `tomaarsen/mpnet-base-nli`, a baseline model trained without using Matryoshka Representation Learning, and `tomaarsen/mpnet-base-nli-matryoshka`, an equivalent model trained using the Matryoshka objective. Both models are trained on the `AllNLI` dataset.

5.5 Embedding Quantization

We experiment with the effect of embedding quantization (None, int8, binary), distinguishing the queries and the document corpus - since the memory burden is on the corpus in practice and we can afford to keep the initial query float precision. We also ablate the compatibility of such optimizations with different search methods: exact i.e. Flat, approximative i.e. IVF and HNSW (Hierarchical Navigable Small Worlds). The exact Flat search is too long at inference and is never used in practice, but it constitutes a good objective baseline. The IVF index is the most standard for approximative search, with a long training time for the preliminary clustering, and a short approximate search time. HNSW indexing does not require training and allows for a precise relatively fast inference. This seems like a good compromise in terms of latency, but is not suited for low-dimensional vectors like ours - thus we do not report the results for it, as its MRR@1000 metric is much worse than IVF - and it will never be used in practice for our corpus.

Lastly, we evaluate the joint interactions of dimension reduction (truncation), quantization, and ML similarity search algorithms on our system and task-specific metrics. All these ablations are done on a subset of 5000 queries from the MSMarco dataset, and with `nlist`=10,000, `nprobe`=30.

6 Evaluation

6.1 System Metrics

Since our end goal is to produce a high-performance end-to-end retrieval system, the evaluation metric we utilize is Mean-Reciprocal-Ranking @ 1000. In this paradigm, given

some set of reference passage-ids *ref*, as well as a query q_i and its predicted relevant passages pred $= \{p_1, ..., p_{1000}\}$, we firstly compute the Reciprocal-Rank @ 1000, or $\frac{1}{\text{rank}}$ (1-indexed) of the first relevant document in *pred*, and average the scores for each document. When comparing two retrieval systems, a better-performing system would rank relevant documents higher, leading to a larger MRR score.

Since our system bottlenecks are also related to memory footprints and search latency, we also record the memory footprint of all corpus + query embedding vectors. This is done by computing num vectors $\times$ embedding_dim $\times$ GB_per_dimension.

Search latency is computed using the wall-clock time utilized to complete all search operations. We acknowledge that this method can be noisy and dependent on the node (e.x. whether a resource is shared) as well as underlying hardware, but it was the best measurement we could feasibly take given the large scale of our experiments and corresponding compute limitations. A potential better evaluation metric would be recording the number of floating-point operations (FLOPS) taken to perform the search operations, though this requires lower-level kernel configuration access (e.x. the Performance Application Performance Interface (PAPI)) that was not available on the clusters we ran experiments on.

Furthermore, we also acknowledge that the time to index a database and potentially train it also factors into the latency of an end-to-end retrieval system. However, indexing operations are typically a one-time cost, and do not particularly scale with the frequency of requests made to a retrieval system. Thus, we believe Queries-Per-Second of search operations is an accurate reflection of our methods' search latencies, and in 6.3.3 only the effective search-time is reported.

6.2 Experimental Setting (Hardware)

Model quantization experiments were run on a NVIDIA RTX A3000 12GB Laptop GPU and a 12th Gen Intel(R) Core(TM) i9-12950HX CPU.

For Dimensionality Reduction experiments, experiments were run across shared node instances on a slurm-managed cluster (babel) that might not have been uniform across experiment instances. Nevertheless, each experiment was run on a node with `64GB RAM` and `4 CPUs`, to partially control the impact of compute limitations on our results. Embeddings were obtained from nodes with 1xV100-32GB.

The experiments for embedding quantization and final ablation studies were done on the same node on a cluster, using the same CPU compute across runs. GPUs (A6000) were only used to generate the embeddings once, and were not involved in the quantization/truncation/search paradigm.

6.3 Results

6.3.1 *Model Quantization*

Results for the GPU and CPU throughput experiments can be found in Figure 3. As expected, looking at the CPU throughput results on the right subplot, we can see that both dynamic int8 and static int8 quantization lead to higher throughput on the CPU compared to the original model. On the other hand, the GPU throughput results did not yield the expected results, as quantization ends up slowing down inference speed (apart from SmoothQuant). This result could be explained by a bottleneck in the GPU quantization implementation, the impact of which is more prevalent as we deal with smaller models. Because quantization is typically done on LLMs, which are very parameter-heavy with model sizes reaching the billion-parameter scale, the large impact of quantization on just the weights themselves masks bottlenecks in the code optimization for the GPU. On the other hand, our Sentence Transformer model is around 96 MB, so the inference is reasonably fast (with respect to one sentence input), and any slowdowns to inference will be prevalent in these contexts.

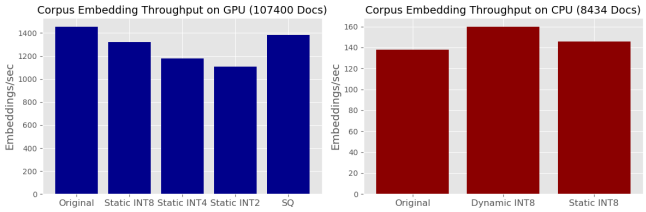


Figure 3. Throughput of quantized models measured in embeddings/sec. GPU results are on the left subplot and CPU results are on the right subplot.

In the right subplot in Figure 4, the memory footprint of each quantized model is shown. We can see that the CPU quantization is more highly optimized as the int8 quantization method has a smaller footprint than int2 quantization on the GPU, supporting our previous claim that our subpar GPU results could be due to a lack of optimization.

Our MRR results in the left subplot are as expected. We see minor degradation with int8/4 quantization and a stark drop in performance with int2 quantization which ends up being too aggressive to produce meaningful results.

Furthermore, we can also see that SmoothQuant's MRR performance is very low. Because SmoothQuant requires a tuning step to calculate the per-channel scaling factors, we ran further experiments to see if the low MRR performance was due to a lack of tuning.

From Table 2, it is clear that SmoothQuant's performance is not improving with more data and not improving by changing whether we calibrate over the queries or the documents.

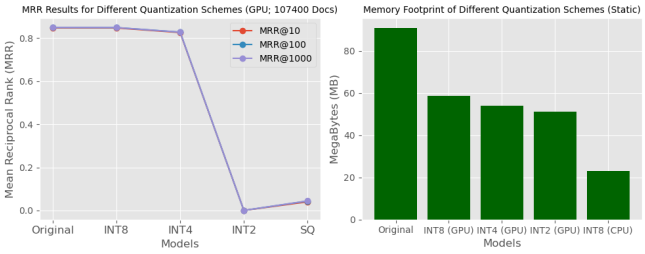


Figure 4. MRR results on MS-Marco @k=10,100,1000 results using different quantization schemes on the GPU can be found on the left subplot. Memory footprints of different quantization schemes can be found on the right subplot.

Document	# of Batches	MRR@1000
Queries	5	0.0447
Queries	50	0.0376
Document	5	0.0219
Document	50	0.0213

Table 2. SmoothQuant MRR results with tuning over different subsets of the data. Batch size is 64 examples.

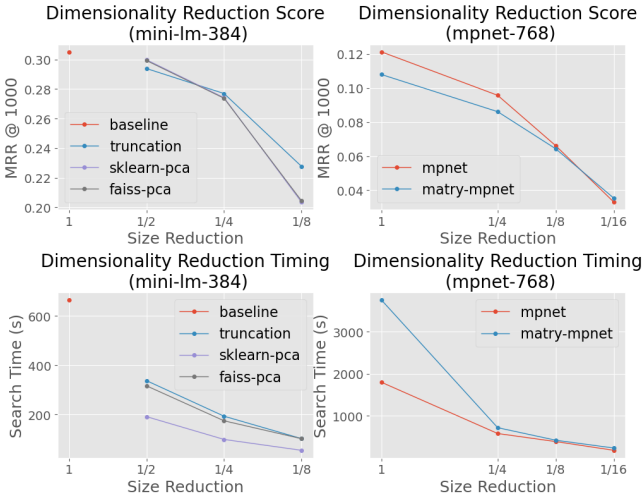


Figure 5. Score and Timing results of dimensionality reduction experiments, using **all** 8.8 million corpus docs and 56k queries. Experiments were run using `nlist`=10,000, `nprobe`=200, `knn`=1000

However, as SmoothQuant's quantization scheme did recover the original inference speed on the GPU, it is possible that the quantization still failed due to suboptimal tuning since these experiments were resource-constrained. As a future direction, we could calibrate the linear modules on the full corpus to get the best per-channel smoothing factors. Additionally, SmoothQuant also introduces a smoothing parameter α that is tuned to be 0.5 for LLMs that essentially encapsulates the migration strength. We could also do a linear search over values of α with respect to MRR performance, as our Sentence Transformer model's weight and activation distributions may not align with the empirical results the SmoothQuant paper observed with LLMs.

6.3.2 *Dimensionality Reduction*

Results for dimensionality reduction experiments can be found in Figure 5. For experiments using `all-MiniLM-L6-v2`, unsurprisingly, the model achieved the highest performance and highest search latency with no size reduction. Up to a size reduction of 1/4, a moderate drop in performance is observed, though significant speed-up gains are achieved, after which the performance drop is too steep to warrant any more speedups.

For Matryoshka Truncation-`mpnet` experiments, baseline performance was already significantly below the performance of `all-MiniLM-L6-v2`. This is likely because the `all-MiniLM-L6-v2` model was trained using a contrastive loss across billions of text pairs and 32 separate benchmark (including `ALL-NLI`), whereas the matryoshka-`mpnet` models were only trained on `ALL-NLI`. Though, the Matryoshka-truncation method unexpectedly underperforms (except for a marginal improvement with severe truncation), indicating that larger-scale Matryoshka Training Paradigms could warrant further research investigation.

6.3.3 *Embedding Quantization and Index Searching*

The results presented in Table 3 confirm our intuitions: the more we reduce the precision of our embeddings, the more memory loss we proportionally gain, but we observe a slight degradation in the final retrieval metric. This is precisely what we obtain using exact search, and this is also overall confirmed with IVF approximative search (except that the MRR@1000 is higher with int8 quantization than without quantization, which is a marginal result of the unpredictability of approximate search).

By chosing IVF approximate search, we can significantly reduce the search latency at the cost of pretraining the index on the corpus, done only once upstream. In the end, choosing a binary quantization for both the queries and the large corpus reduces the memory by 32, and preserves a decent final MRR@1000, while alleviating the approximative search latency. We hypothesize that a hybrid quantization approach would be beneficial and complete further ablation studies in next section.

6.3.4 *Final Ablation Studies*

Following the additional ablations reported in Table 4:

Query	Corpus	Memory (GB)	FAISS-Index	Search (s)	MRR@1000
float	float	13.59	Flat	1136.5	0.373
float	**float**	**13.59**	**IVF**	**112.7**	**0.343**
int	int	3.4	Flat	1099.0	0.366
int	**int**	**3.4**	**IVF**	**54.7**	**0.347**
bin	bin	0.42	BinFlat	301.6	0.333
bin	**bin**	**0.42**	**BinIVF**	**7.4**	**0.303**

Table 3. Impact of query/corpus quantization, and Index search on the Memory, Speed, and MRR. Setting: 5000 queries from MSMarco dev set, all 8.8 million corpus docs. Index time not reported for IVF (`nlist`=500,`nprobe`=10); long but only done once in practice.

Query	Corpus	Memory (GB)	FAISS-Index	Search (s)	MRR@1000
float/1	int/1	3.4	Flat	1104.5	0.372
int/1	int/1	3.4	IVF	1116.2	0.372
float/1	int/1	3.4	IVF	21.2	7.727 e-6
float/2	int/2	1.7	Flat	658.2	0.346
int/2	int/2	0.85	IVF	145.0	0.351
float/4	int/4	0.85	Flat	459.0	0.292
int/4	**int/4**	**0.85**	**IVF**	**131.6**	**0.330**
int/8	int/8	0.42	IVF	76.2	0.259
int/16	int/16	0.21	IVF	30.4	0.101

Table 4. Impact of different hybrid quantizations, truncations and search algorithms. Setting: same as 3

Our hypothesis is not confirmed, since a hybrid quantization (float for the queries, int8 for the corpus) does not perform significantly better than the symmetric int8-int8 quantization. In particular, this hybrid quantization is highly not compatible with an IVF index, which is the one used in practice. Further research could confirm this with a more thorough full-reranking benchmark (as this was on a subset) - as we believe the idea is promising. Particularly, for a reranking approach, we can use exact float and int embeddings on the smaller retrieved subset.

By fixing int8 quantization for both queries and embeddings, we see that truncating the embeddings by a factor 4 still retains an impressive MRR@1000 of 0.33, which is similar to the baseline we had without any truncation, resulting in a reduction of the memory footprint by 16 and an equivalent latency compared to the original baseline without any optimizations.

7 Conclusion

There are many opportunities to optimize retrieval-augmented systems, and our findings are summarized as follows.

Model Quantization can be an effective method for reducing the latency of generating embeddings and model memory footprint with little to no performance degradation as seen in int8 CPU quantization. However, as seen with the GPU quantization results, it is important that the quantization method is optimized as bottlenecks in the model inference are more impactful in smaller size models like `sentence-transformers`.

Dimensionality Reduction methods can jointly provide significant memory and speed-up gains (up to 1/4, realistically), though the quantity of gains is dependent on how much performance degradation is allowed. Embedding Quantization (separately and jointly with dimensionality reduction) can easily result in a reduction by 16 of the memory footprint while preserving retrieval performance. Our study also sheds light on the overlooked link between embedding reduction techniques and indexing/search methods, as the usual approximate search IVF seems incompatible with the promising float-int8 hybrid query-corpus quantization. However, the latter is still promising in the context of retrieve-and-rerank two-step pipelines, where exact similarity-reranking on a small subset of candidates is possible. More work at the intersection of embedding quantization (systems) and approximate search algorithms (ML) is thus warranted, with more benchmarks across different embedding models and involving cross-encoders for reranking.

References

Bajaj, P., Campos, D., Craswell, N., Deng, L., Gao, J., Liu, X., Majumder, R., McNamara, A., Mitra, B., Nguyen, T., Rosenberg, M., Song, X., Stoica, A., Tiwary, S., and

Wang, T. Ms marco: A human generated machine reading comprehension dataset, 2018.

Dettmers, T., Lewis, M., Belkada, Y., and Zettlemoyer, L. Llm.int8(): 8-bit matrix multiplication for transformers at scale, 2022.

Douze, M., Guzhva, A., Deng, C., Johnson, J., Szilvasy, G., Mazaré, P.-E., Lomeli, M., Hosseini, L., and Jégou, H. The faiss library. 2024.

Frantar, E., Ashkboos, S., Hoefler, T., and Alistarh, D. Gptq: Accurate post-training quantization for generative pre-trained transformers, 2023.

Kusupati, A., Bhatt, G., Rege, A., Wallingford, M., Sinha, A., Ramanujan, V., Howard-Snyder, W., Chen, K., Kakade, S., Jain, P., and Farhadi, A. Matryoshka representation learning, 2024.

Lewis, P., Perez, E., Piktus, A., Petroni, F., Karpukhin, V., Goyal, N., Küttler, H., Lewis, M., tau Yih, W., Rocktäschel, T., Riedel, S., and Kiela, D. Retrieval-augmented generation for knowledge-intensive nlp tasks, 2021.

Ma, S., Wang, H., Ma, L., Wang, L., Wang, W., Huang, S., Dong, L., Wang, R., Xue, J., and Wei, F. The era of 1-bit llms: All large language models are in 1.58 bits, 2024.

Reimers, N. and Gurevych, I. Sentence-bert: Sentence embeddings using siamese bert-networks. In *Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing*. Association for Computational Linguistics, 11 2019. URL https://arxiv.org/abs/1908.10084.

Spotify. annoy. https://github.com/spotify/annoy, 2015.

Xiao, G., Lin, J., Seznec, M., Wu, H., Demouth, J., and Han, S. Smoothquant: Accurate and efficient post-training quantization for large language models, 2024.

Zhang, G., Zhou, Y., and Bollegala, D. Evaluating unsupervised dimensionality reduction methods for pretrained sentence embeddings, 2024.

A APPENDIX

Experiment	Experient Id	Reduction Latency	Embedding Time	KNN Latency (Train/Search)	Embedding Footprint	MRR@1000
Base-384	0	-	5036.14 s	2194.05 s / 655.10 s	13.67GB	0.3051
Trunc-192	1	-	-	1261.69 s / 337.54 s	6.83GB	0.2938
Trunc-96	1	-	-	846.03 s / 193.30 s	3.42GB	0.2769
Trunc-48	1	-	-	635.38 s / 102.25 s	1.71GB	0.2276
SK-PCA-192	2	4427.83 s	-	746.58 s / 191.47 s	6.83GB	0.2997
SK-PCA-96	2	2496.34 s	-	482.22 s / 98.96 s	3.42GB	0.2741
SK-PCA-48	2	1319.79 s	-	390.99 s / 54.79 s	1.71GB	0.2034
FAISS-PCA-192	3	1.0675 s	-	1252.11 / 316.55 s	6.83GB	0.2992
FAISS-PCA-96	3	1.0680 s	-	836.16 / 174.61 s	3.42GB	0.2737
FAISS-PCA-48	3	1.4270 s	-	743.98 / 101.94 s	1.71GB	0.2043
MPNet-768	4	-	12208.05 s	4095.79 s / 1796.51 s	27.34GB	0.1212
MPNet-192	5	-	-	1286.79 s / 578.82 s	6.83GB	0.0957
MPNet-96	5		-	876.67 s / 386.75 s	3.42GB	0.0661
MPNet-48	5	-	-	661.79 s / 179.28 s	1.71GB	0.0330
MPNet-Matry-768	6	-	18151.66 s	7222.41 s / 3751.51 s	27.34GB	0.1079
MPNet-Matry-192	7	-	-	2049.24 s / 717.53 s	6.83GB	0.0861
MPNet-Matry-96	7	-	-	1369.70 s / 419.45 s	3.42GB	0.0643
MPNet-Matry-48	7	-	-	1019.28 s / 233.27 s	1.71GB	0.0353

Table 5. Detailed Results from Dimensionality Reduction experiments. Experiments of the same ID were run consecutively on the same node. Experiments without an embedding time use the same embeddings as the experiments above. Embeddings were obtained on nodes with 1xV100-32GB. KNN experiments were run on nodes with 64GB RAM and 4xCPUs